

Crackmes.one - Azure

Author: msym

This is a very brief write-up of how we can determine the login credentials for the program *azure.exe* (challenge 6962a42080f66abfe18577f0 on crackmes.one) expeditiously.

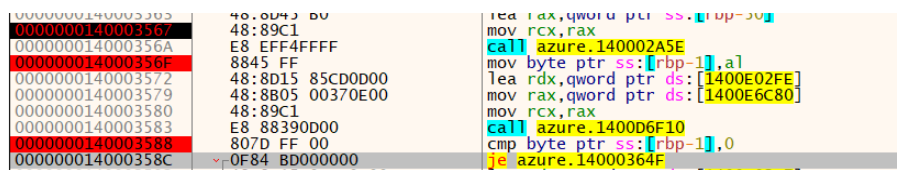
Because this crackme is quite large and possibly quite tricky, it is likely that we will benefit from analysing the crackme both statically and dynamically, in which case it is recommended to disable Address Space Layout Randomisation (ASLR) (using, for example, CFF explorer) - this crackme has ASLR enabled. Once we are done with this, we can run the program and attach a debugger such as x64dbg (the program is 64-bit Windows).

We can start off by keying in some random user and password (I keyed in `msymm` and `12345`). Stepping out multiple times brings us to the address `140003506` (from `ntdll.dll` and other DLLs; roughly 7 times -> `1400D5434` -> `140003506`).

Scrolling down a little, we will notice the instructions:

1. `1400035BE: lea rdx, qword ptr ds:[1400E0398]`, and x64dbg indicates the presence of the string `ACCESS GRANTED`; and
2. `14000367B: lea rdx, qword ptr ds:[1400E0498]`, and x64dbg indicates the presence of the string `ACCESS DENIED`.

Notice (by stepping over the function calls, or just by careful observation) that which message is displayed depends on whether the jump is taken at the instruction `14000358C: je azure.14000364F`, which boils down to the previous instruction `cmp byte ptr ss:[rbp-1], 0`. The screenshot below shows these crucial instructions.



The value stored at `rbp-1` is the value in the AL register immediately after the function call `call azure.140002A5E`. This suggests that the validation of the inputted credentials likely happen within this function call. We can verify dynamically that the RCX and RDX registers respectively hold an address storing a pointer the user and password inputs.

0000000140003535	C645 FF 00	mov byte ptr ss:[rbp-1],^		
0000000140003539	48:8D55 D0	lea rdx,qword ptr ss:[rbp-1]		Hide FPU
000000014000353D	48:8D45 B0	lea rax,qword ptr ss:[rbp-1]		
0000000140003541	48:89C1	mov rcx,rax	RAX	00000000005FFE20 &"msymm"
0000000140003544	E8 B4F3FFFF	call azure.1400028F0	RBX	0000000140111050 azure.0000000000000000
0000000140003549	8845 FE	mov byte ptr ss:[rbp-2],	RCX	00000000005FFE20 &"msymm"
000000014000354C	48:8D55 D0	lea rdx,qword ptr ss:[rbp-2]	RDY	00000000005FFE40 &"12345"
0000000140003550	48:8D45 B0	lea rax,qword ptr ss:[rbp-2]	RBP	00000000005FFE70
0000000140003554	48:89C1	mov rcx,rax	RSP	00000000005FFE00 &"0b_"
0000000140003557	E8 B0F4FFFF	call azure.140002A0C	RSI	0000000000000001
000000014000355C	8845 FD	mov byte ptr ss:[rbp-3],	RDI	000000000000002C
000000014000355F	48:8D55 D0	lea rdx,qword ptr ss:[rbp-3]		
0000000140003563	48:8D45 B0	lea rax,qword ptr ss:[rbp-3]	R8	0000000000000035 '5'
0000000140003566	48:89C1	mov rcx,rax	R9	00007FF84AC80000 "mzE"
000000014000356A	E8 EFF4FFFF	call azure.140002A5E	R10	00000000005FFE50 "12345"
000000014000356F	8845 FF	mov byte ptr ss:[rbp-1],	R11	00000000005FFDD5 "12345"
0000000140003574	48:89C1	mov rcx,rax		

Because the number of instructions in FUN_140002A5E made it look quite daunting, it may be worthwhile trying to analyse it together with the help of Ghidra's decompiler.

At the end of FUN_140002A5E , we observe that the return value of the function is 1 (i.e validation is successful) provided a certain set of local variables (Ghidra labelled them as local_19 , local_29 , ..., local_6f , and I am too lazy to rename them...) are non-zero.

```

if (((local_19 == '\0') || (local_29 == '\0')) ||
    ((local_6d == '\0' || ((local_6e == '\0' || (local_6f == '\0')) || (!local_79)))))) {
    uVar8 = 0;
}
else {
    uVar8 = 1;
}
_ZNSt6vectorIhSaIhEED1Ev(local_198);
_ZN2VMD1Ev(local_478);
}
_ZNSt6vectorIhSaIhEED1Ev(local_178);
_ZNSt6vectorIhSaIhEED1Ev(local_158);
_ZNSt6vectorIhSaIhEED1Ev(local_138);
_ZNSt6vectorIhSaIhEED1Ev(local_118);
return uVar8;

```

We will have to track the conditions under which these local variables are set to a non-zero value. Scrolling up, we will see, for example, the following if-else statements which determine the value of local_6d and local_6e .

```

}
if (((local_40 == local_58) && (local_44 == local_5c)) && (local_48 == local_60)) {
    local_6d = '\x01';
}
else {
    local_6d = '\0';
}
if (((local_4c == local_64) && (local_50 == local_68)) && (local_54 == local_6c)) {
    local_6e = '\x01';
}
else {
    local_6e = '\0';
}

```

They depend on some other local variables being equal, and scrolling up even more, we see that these local variables local_40 , local_44 ... are the images of other local variables under three different functions.

```

else {
    local_40 = TRANSFORM1(local_158);
    local_44 = TRANSFORM2(local_158);
    local_48 = TRANSFORM3(local_158);
    local_4c = TRANSFORM1(local_178);
    local_50 = TRANSFORM2(local_178);
    local_54 = TRANSFORM3(local_178);
    local_58 = TRANSFORM1(local_118);
    local_5c = TRANSFORM2(local_118);
    local_60 = TRANSFORM3(local_118);
    local_64 = TRANSFORM1(local_138);
    local_68 = TRANSFORM2(local_138);
    local_6c = TRANSFORM3(local_138);
}

```

Ghidra gave the functions awfully long and unintuitive names, so I renamed them as TRANSFORM1, TRANSFORM2 and TRANSFORM3. Basically, this means that if local_118 = local_158 and local_138 = local_178, the checks in the above if-else statements will succeed. So let's examine what values are held in these local variables.

Scrolling even further up, it seems that local_158 and local_178 are holding outputs of the transformation of the user and password inputs, (under a function which I renamed TRANSFORM for readability).

```

_ZNSt7__cxx112basic_stringIcSt11char_traitsIcESaIcEEC1IS3_EEPKcRKKS3_(local_f8,"svidnet");
TRANSFORM(local_118,local_f8,0x42);
_ZNSt7__cxx112basic_stringIcSt11char_traitsIcESaIcEEED1Ev(local_f8);
_ZNSt15__new_allocatorIcED2Ev();
bVar3 = local_3a;
local_98 = &local_99;
_ZNSt7__cxx112basic_stringIcSt11char_traitsIcESaIcEEC1IS3_EEPKcRKKS3_(
    (local_c8,"ERR//NO_SUCH_FILE_OR_DIR000xsssh");
TRANSFORM(local_138,local_c8,bVar3);
_ZNSt7__cxx112basic_stringIcSt11char_traitsIcESaIcEEED1Ev(local_c8);
_ZNSt15__new_allocatorIcED2Ev();
TRANSFORM(local_158,param_1,local_39);
TRANSFORM(local_178,param_2,local_3a);

```

It also seems that local_f8 and local_c8 respectively hold the values svidnet and ERR//NO_SUCH_FILE_OR_DIR000xsssh. We can verify this in x64dbg. The third parameter passed to the TRANSFORM function are the same for the calls made with output in local_118 and local_158 (local_39 = 0x42), and with output in local_138 and local_178 (local_3a = bVar3 = 0x7e).

NOTE: When we say a local variable holds a certain value (e.g. the inputted password), what we mean, to be precise, is that the variable holds an address which holds a pointer to that value.

It seems very promising that the user is svidnet and the password is ERR//NO_SUCH_FILE_OR_DIR000xsssh. It is not difficult to check that with these credentials, the other checks (local_19 != 0, local_29 != 0 etc) will also succeed.

Let's determine if our analysis is correct.

```
|| Try to crack. Do not try to guess =>
||
||
└─┘
- USER
-> svidnet
- PASS
-> ERR//NO_SUCH_FILE_OR_DIR000xssh
*] checking...
ACCESS GRANTED!
Welcome, svidnet!
system initialized successfully.
press Enter to exit...
```

Indeed, we conclude that the credentials are:

USER: svidnet

PASS: ERR//NO_SUCH_FILE_OR_DIR000xssh .